رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

# Data Analysis to Answer Questions

Session 1

# Agenda

- Defining Data Analysis Questions
- Applying Sorts & Filters Using Python

# Define Data-Analysis Questions

# Defining Data-Analysis Questions

**Introduction:**

- Crafting **clear, actionable data-analysis questions** is essential for informed decision-making.
- It's about **translating objectives** into tangible queries that data can address

**Expansion:**

- Effective data analysis starts with **well-defined questions** that reflect the desired outcomes.
- The better scoped a question is, the more likely the analysis will yield useful insights.

# Example Scenario: Poorly Defined vs. Well-Defined Questions

## Poorly Defined Question:

"Why are our sales down?"

**Outcome:** This question is vague and lacks specifics, making it difficult to identify the root cause. Analysis may yield broad insights but won't lead to actionable steps.

## Well-Defined Question:

"What factors contributed to a 20% decrease in sales for our electronics category in Q2 compared to Q1?"

**Outcome:** This question specifies the category (electronics), the timeframe (Q2 vs. Q1), and the exact issue (20% decrease). Analysis can focus on relevant data, such as customer feedback, competitor pricing, and marketing efforts, leading to targeted strategies for improvement.

# Key Steps for Defining Data-Analysis Questions

## 1- Define Objectives :

- Understand **challenges**, **decisions**, and **desired outcomes**.
- Engage stakeholders to clarify **goals** and **expectations**.
- Objectives should be **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound).

# SMART Criteria for Setting Objectives

The SMART criteria help ensure that objectives are well-defined and actionable. Each component serves a specific purpose in guiding effective goal-setting.

**S - Specific**

- **Explanation:** Objectives should be clear and specific, answering the who, what, where, when, and why. A specific goal eliminates ambiguity and provides a clear direction.
- **Context:** For instance, instead of saying "Increase sales," a specific objective would be "Increase online sales of product X by 15% within the next quarter.“

**M - Measurable**

- **Explanation:** There should be measurable indicators to track progress and determine when the objective has been achieved. This helps in evaluating the success of efforts.
- **Context:** An objective like "Improve customer satisfaction" can be measured through surveys or Net Promoter Scores (NPS), e.g., "Achieve an NPS of 50 by the end of the year."

# SMART Criteria for Setting Objectives

**A – Achievable**

- **Explanation:** The objective should be realistic and attainable, considering available resources and constraints. This criterion ensures that the goal is within reach, which boosts motivation.
- **Context:** Instead of aiming to "double sales in one month," a more achievable goal might be "Increase monthly sales by 10% over the next six months.“

**R – Relevant**

- **Explanation:** Objectives must align with broader business goals and be relevant to the organization’s mission. This ensures that the efforts contribute meaningfully to overall success.
- **Context:** A relevant objective might be "Launch a new marketing campaign for product Y to target millennials," which aligns with a company's strategy to capture a younger demographic.

# SMART Criteria for Setting Objectives

**T - Time-bound**

- **Explanation:** A deadline should be established for achieving the objective, creating urgency and prompting action. This prevents objectives from being open-ended and helps prioritize tasks.
- **Context:** An objective like "Reduce customer complaints by 20% by the end of Q2" gives a clear timeframe, motivating teams to act within a specified period.

# Key Steps for Defining Data-Analysis Questions

**Example:**

- Objective: "Increase customer retention by 15% over the next quarter”.

- **SMART**:
- **Specific**: Increase customer retention.
- **Measurable**: 15% increase.
- **Achievable**: Focus on high-priority customers to ensure the goal is realistic.
- **Relevant**: Linked to overall business goals of improving customer retention.
- **Time-bound**: To be achieved by the end of the quarter.

# Key Steps for Defining Data-Analysis Questions

## 2- Assess Data:

- Evaluate **available data**, its sources, quality, and relevance.
- Consider the **variety**, **volume**, and **velocity** of data to determine its usability.
- Identify any **gaps** or limitations in the data that may impact the analysis.

**Example:**

- **Available data**: Customer demographic info, transaction history, churn rates.
- **Limitation**: No data on customer satisfaction post-purchase.

# Key Steps for Defining Data-Analysis Questions

## 3- Craft Questions:

- Create specific, **measurable**, and **actionable** queries aligned with objectives and data.
- Questions should focus on the **variables** that are most relevant to the objectives.
- Consider the level of **granularity** needed and any potential biases in the data.

**Example Questions:**

- How does **customer age** influence purchase frequency?
- What is the impact of **loyalty programs** on customer retention rates?
- Do **sales promotions** increase the likelihood of returning customers?

# Key Steps for Defining Data-Analysis Questions

## 4- Iterate and Adapt:

- Maintain flexibility for **evolving insights** and changing circumstances.
- Regularly review and refine **data-analysis questions** as new information emerges.
- Adapt questions based on **feedback**, emerging trends, or shifts in organizational priorities.

**Example:**

- Initial question: “What is the best time of year to run promotions?
- ”Revised question: “What time of year do promotions lead to the highest return customer rate in specific product categories?”

# Strategies for Identifying Actionable Questions

- Start by breaking down **business goals** into measurable outcomes.
- **Collaborate with stakeholders** to prioritize questions.
- Assess **available datasets** to ensure the data is sufficient for analysis.

**Example:**

- For a retail company:
  **Question:** How does customer age affect purchasing habits in different regions?
  **Data:** Customer demographics, sales data.
- For a hospital:
  **Question:** Does patient wait time correlate with satisfaction scores?
  **Data:** Patient records, and feedback forms.

# Scoping Questions Based on Available Data

- Evaluate the quality of **data sources** and ensure the data matches the scope of the question.
- For larger questions, consider **segmenting** them into smaller, more specific questions.

**Tips for Scoping:**

- Ensure the question can be answered with **existing datasets**.
- Be clear on what **data format** (e.g., structured, unstructured) you have.
- Align questions with **actionable insights** that can influence decision-making.

# Well-Scoped Questions Example

- **Scenario 1:** E-commerce Platform:

  **Objective:** Increase sales by improving the customer experience.
  **Well-scoped question:** Which products are abandoned in carts most frequently, and how do promotions affect cart recovery rates?

- **Scenario 2:** Healthcare Provider

  **Objective**: Reduce patient readmission rates.
  **Well-scoped question:** What patterns in patient history are most predictive of hospital readmission within 30 days?

# Iterating and Adapting Questions

- Maintain **flexibility** for evolving insights and changing circumstances.
- Regularly **review and refine** questions as new information emerges.
- Adapt questions based on **feedback**, emerging trends, or shifts in priorities.

# Iterating and Adapting Questions

**Example Scenario**

**Initial Question:**
•"What are the sales trends for Product A over the past year?“

**Findings:**
•After analyzing the initial data, analysts discover that a recent marketing campaign significantly increased sales in specific demographics, such as millennials.

**Adapted Question:**
•"How did the marketing campaign impact the sales trends of Product A among millennials over the past year?“

**Impact of the Change:**
•By adapting the question to focus on the impact of the marketing campaign on a specific demographic, analysts can uncover insights that help the marketing team understand the effectiveness of their strategies. This leads to data-driven decisions, such as optimizing future campaigns for similar audiences, which could further enhance sales and customer engagement.

# Applying Sorts & Filters Using Python

# Basic Sorting Techniques

- **Single Column Sort:** Use ***sort_values()*** to organize data based on one column.

```
import pandas as pd

# Creating DataFrame
data = {'Name': ['John', 'Anna', 'James', 'Linda'],
        'Age': [28, 22, 35, 32],
        'Salary': [50000, 62000, 55000, 48000]}
df = pd.DataFrame(data)

# Sorting by age
sorted_df = df.sort_values(by='Age', ascending=True)
sorted_df
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Anna | 22 | 62000 |
| 0 | John | 28 | 50000 |
| 3 | Linda | 32 | 48000 |
| 2 | James | 35 | 55000 |

# Basic Sorting Techniques

- **Multiple Columns Sort:** Extend sorting to multiple columns to refine your data views.

```
# Sorting by age, then salary
sorted_df = df.sort_values(by=['Age', 'Salary'], ascending=[False, True])
sorted_df
```

| | Name | Age | Salary |
|---|---|---|---|
| 2 | James | 35 | 55000 |
| 3 | Linda | 32 | 48000 |
| 0 | John | 28 | 50000 |
| 1 | Anna | 22 | 62000 |

# Basic Sorting Techniques

**Task:** Sort a provided dataset by salary in descending order.

# Advanced Sorting Techniques

- **Sort by Index:** Utilize *sort_index()* for organizing data based on DataFrame index.
- **Conditional Sorting:** Apply conditions to sorting, such as custom functions.

```
# Sorting by index
index_sorted_df = df.sort_index(ascending=False)
index_sorted_df
```

| | Name | Age | Salary |
|---|---|---|---|
| 3 | Linda | 32 | 48000 |
| 2 | James | 35 | 55000 |
| 1 | Anna | 22 | 62000 |
| 0 | John | 28 | 50000 |

```
# Custom sort using lambda
custom_sort = df.sort_values(by='Name', key=lambda col: col.str.lower())
custom_sort
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Anna | 22 | 62000 |
| 2 | James | 35 | 55000 |
| 0 | John | 28 | 50000 |
| 3 | Linda | 32 | 48000 |

# Advanced Sorting Techniques

**Task:** Sort a dataset first by a categorical column in descending order, then by a numerical column.

# Basic Filtering Techniques

- **Simple Condition Filters:** Apply conditions directly to DataFrame columns.
- **Logical Operators:** Combine multiple conditions using AND (&), OR (|).

```
# Simple filter
young_professionals = df[df['Age'] < 30]
young_professionals
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | John | 28 | 50000 |
| 1 | Anna | 22 | 62000 |

```
# Combined conditions
high_earning_young = df[(df['Salary'] > 50000) & (df['Age'] < 30)]
high_earning_young
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Anna | 22 | 62000 |

# Basic Filtering Techniques

**Task:** Filter for employees who earn more than $50,000 and are under 30 years old.

# Advanced Filtering Techniques

Filtering Using **isin**() is useful when you want to filter rows where a column's value is in a list of specific values.
Filtering with **str.contains**() for Text Matching This is used when you want to filter rows based on whether a string contains a specific substring.

```
# Filter rows where the 'Name' is either 'Anna' or 'John'
filtered_df = df[df['Name'].isin(['Anna', 'John'])]
filtered_df
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | John | 28 | 50000 |
| 1 | Anna | 22 | 62000 |

```
# Filter rows where 'Name' contains the letter 'a'
filtered_df = df[df['Name'].str.contains('a', case=False)]
filtered_df
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Anna | 22 | 62000 |
| 2 | James | 35 | 55000 |
| 3 | Linda | 32 | 48000 |

# Advanced Filtering Techniques

- **Using query() Method:** Perform complex filters using a string expression.
- **String Methods in Filters:** Use string methods for pattern-based filtering.

```
# Using query
query_filtered = df.query('Salary > 50000 and Age < 30')
query_filtered
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Anna | 22 | 62000 |

```
# Filtering with string methods
j_names = df[df['Name'].str.startswith('J')]
j_names
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | John | 28 | 50000 |
| 2 | James | 35 | 55000 |

# Advanced Filtering Techniques

- **Filtering with between()** filters data between two values, inclusive.
- **Filtering Using a Custom Function (apply)** You can also apply a custom function to filter rows based on more complex logic.

```
# Filter rows where 'Age' is between 25 and 35
filtered_df = df[df['Age'].between(25, 35)]
filtered_df
```

| | Name | Age | Salary |
|---|---|---|---|
| 0 | John | 28 | 50000 |
| 2 | James | 35 | 55000 |
| 3 | Linda | 32 | 48000 |

```
# Define custom function to filter rows where 'Salary' is above average
def filter_high_salary(row):
    return row['Salary'] > df['Salary'].mean()

# Apply custom function
filtered_df = df[df.apply(filter_high_salary, axis=1)]
filtered_df
```

| | Name | Age | Salary |
|---|---|---|---|
| 1 | Anna | 22 | 62000 |
| 2 | James | 35 | 55000 |

# Advanced Filtering Techniques

**Task:** Apply query() to filter employees whose age is less than the average age of the workforce.

# Let’s Practice

# E-commerce Transactions Hand-on Exercise

**Use this simulated data that represents transactions from an online retail store.**

```
data = {
    'CustomerID': [12345, 12346, 12347, 12348, 12349, 12350, 12351, 12352, 12353, 12354],
    'OrderID': [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010],
    'Product': ['T-shirt', 'Bicycle', 'Water bottle', 'Backpack', 'Sunglasses', 'Notebook', 'Tablet', 'Smartphone',
'Camera', 'Headphones'],
    'Quantity': [2, 1, 5, 2, 1, 4, 1, 1, 1, 2],
    'UnitPrice': [15.00, 200.00, 8.00, 45.00, 25.00, 12.00, 350.00, 700.00, 250.00, 75.00],
    'PurchaseDate': ['2022-07-15 08:30:00', '2022-07-15 09:00:00', '2022-07-16 10:00:00',
                     '2022-07-16 11:30:00', '2022-07-17 12:45:00', '2022-07-18 13:00:00',
                     '2022-07-19 14:25:00', '2022-07-20 15:45:00', '2022-07-21 16:00:00',
                     '2022-07-22 17:35:00'],
    'Country': ['USA', 'Canada', 'UK', 'USA', 'Australia', 'India', 'Germany', 'France', 'Spain', 'Italy']
}
df = pd.DataFrame(data)
```

# E-commerce Transactions Hand-on Exercise

**Tasks:**

1- Identify the Top 10% Customers.
2- Analyze Sales Events:

- Filter transactions that occurred during major sales events, like Black Friday or annual sales.
- Observe purchasing patterns during these periods to identify popular products or buying trends.

# E-commerce Transactions Hand-on Exercise

```
# Calculate total spend per customer

df['TotalSpend'] = df['Quantity'] * df['UnitPrice']
top_spenders_threshold = df['TotalSpend'].quantile(0.90)
top_spenders = df[df['TotalSpend'] >= top_spenders_threshold]
print(top_spenders)
```

```
# Assuming sales events dates are known

sales_events_dates = pd.to_datetime(['2022-11-25', '2022-11-28'])  # Example dates for Black Friday and Cyber Monday
df_sales_events = df[df['PurchaseDate'].dt.floor('D').isin(sales_events_dates)]
print(df_sales_events)
```

# Any Questions ?

# Thank You